

**Università degli Studi di Salerno**

**Corso di Ingegneria del Software**

**Easy Work Management System**

**Object Design Document**

**Versione 1.0**



**E.W.M.S.**

**Easy Work Management System**

Data: 15/12/2025

|                                       |                  |
|---------------------------------------|------------------|
| Progetto: Easy Work Management System | Versione: 1.0    |
| Documento: Object Design Document     | Data: 15/12/2025 |

**Coordinatore del progetto:**

| Nome                | Matricola  |
|---------------------|------------|
| Giovanni Robustelli | 0512121282 |
| Vincenzo Mauro      | 0512119080 |

**Partecipanti:**

| Nome                | Matricola  |
|---------------------|------------|
| Giovanni Robustelli | 0512121282 |
| Vincenzo Mauro      | 0512119080 |
|                     |            |
|                     |            |
|                     |            |
|                     |            |

|                    |                                     |
|--------------------|-------------------------------------|
| <b>Scritto da:</b> | Vincenzo Mauro, Giovanni Robustelli |
|--------------------|-------------------------------------|

**Revision History**

| Data       | Versione | Descrizione                                                  | Autore                                |
|------------|----------|--------------------------------------------------------------|---------------------------------------|
| 15/12/2025 | 1.0      | Assemblaggio delle singole parti sviluppate dai partecipanti | Giovanni Robustelli<br>Vincenzo Mauro |
|            |          |                                                              |                                       |
|            |          |                                                              |                                       |
|            |          |                                                              |                                       |
|            |          |                                                              |                                       |
|            |          |                                                              |                                       |
|            |          |                                                              |                                       |

# Indice

|     |                                                  |    |
|-----|--------------------------------------------------|----|
| 1.  | Introduzione .....                               | 4  |
| 1.1 | Object design trade-offs .....                   | 4  |
| 1.2 | Interface documentation guidelines .....         | 5  |
| 2.  | Pacchetti .....                                  | 7  |
| 2.1 | Decomposizione dei sottosistemi in package ..... | 7  |
| 2.3 | Dipendenze tra package.....                      | 9  |
| 3.  | Class interfaces .....                           | 10 |
| 3.1 | AccessManagement.....                            | 10 |
| 3.2 | AccountManagement .....                          | 11 |
| 3.3 | TaskManagement .....                             | 15 |
| 3.4 | NotificheManagement.....                         | 20 |
| 3.5 | PersistanceManagement .....                      | 21 |

# 1. Introduzione

E.W.M.S. è una piattaforma di gestione dei task progettata per facilitare la comunicazione tra dipendenti e supervisori in contesti operativi caratterizzati da un elevato volume di attività, come magazzini, centri logistici e reparti industriali. Il sistema punta a centralizzare l'assegnazione dei compiti e il monitoraggio dello stato delle attività, riducendo i tempi di coordinamento e aumentando l'efficienza e la trasparenza organizzativa.

Questo documento serve a chiarire e presentare le decisioni riguardanti l'object design su cui ci si baserà per costruire il prodotto E.W.M.S., oltre che a definire la decomposizione del sistema in package e le loro responsabilità, il mapping tra requisiti funzionali e oggetti di design, nonché la relazione tra questi ultimi e i pattern adottati.

Inoltre, verranno mostrate le strategie di persistenza e gli aspetti che determinano la qualità del design, tutto questo per favorire l'implementazione e lo sviluppo del sistema.

## 1.1 Object design trade-offs

### **Complessità vs. manutenibilità**

È nel nostro interesse utilizzare un modello ad oggetti semplice e composto da poche classi centrali suddivise in maniera non eccessiva, riducendo il rischio di over-design e accelerando il processo di sviluppo.

Questa semplificazione tende a ridurre la capacità del modello di rappresentare esaurientemente delle operazioni/situazioni specifiche che però non si applica sullo scope del nostro prodotto, poiché sono previste astrazioni tali da coprire la maggior parte degli aspetti richiesti da un classico task manager.

Questo si traduce in una strutturazione del codice complessa per i programmatori ma facile da applicare e comprendere, in modo da adattare quanto più possibile il prodotto ai design goals specificati nel SDD.

Per quanto riguarda la manutenzione del sistema, se ne occuperà il team di sviluppo originale (vedi RAD, 3.3.4 Supportabilità – Manutenibilità), quindi all'azienda che adotterà il prodotto non verrà richiesto impegno aggiuntivo.

## Buy vs. build

Tutti i componenti principali del sistema saranno implementati internamente, assicurando il controllo completo sulla logica di business, saranno invece adottate librerie standard esterne per le funzionalità infrastrutturali, riducendo i costi di sviluppo e i rischi di errori da risolvere con la manutenzione.

## Memoria vs. performance

Il sistema si basa su strutture dati semplici con un caricamento dei dati “on-demand”, favorendo i tempi di risposta rispetto ad una memorizzazione temporanea prolungata, e quindi le performance.

Questa scelta minimizza il consumo di memoria e aumenta la flessibilità del sistema a fronte di molteplici richieste di servizio e interrogazione al database.

## Ottimizzazione vs. usabilità

Il sistema è ideato per essere user-friendly, in modo da favorire l’accessibilità e l’usabilità dello stesso, senza però sacrificare gli aspetti tecnici come performance, scalabilità e affidabilità.

Inoltre, utilizzando interfacce generiche, vengono privilegiate anche l’estendibilità e la sostituibilità delle componenti del sistema.

Tuttavia, questa scelta limita le prestazioni ottenute rispetto ad un’implementazione focalizzata sulla performance.

## 1.2 Interface documentation guidelines

Al fine di garantire coerenza, qualità e comprensibilità del progetto, l’object design segue le seguenti linee guida:

### Object naming conventions:

- le **classi** utilizzeranno nomi abbastanza descrittivi della loro funzione e seguiranno il metodo di scrittura *PascalCase*.
- I **metodi** seguiranno la stessa metodica delle classi, descrivendo un’azione nel loro nome, scritto in *camelCase*.
- Gli **attributi** rappresenteranno proprietà dell’oggetto, utilizzando sostantivi brevi e significativi.

- Ogni **package** rappresenterà un ambito funzionale del sistema (Task, Notifiche, Repository)

### **Boundary Cases:**

verrà riposta particolare attenzione nei casi limite, come input nulli, inconsistenza degli oggetti, task non assegnati o già completati, connessioni interrotte.

Naturalmente le classi prevedono e definiscono comportamenti prevedibili e documentati per questi casi.

### **Exception Handling:**

le eccezioni non sono previste come meccanismo di controllo del flusso della fruibilità del sistema ma solo come segnalazioni di condizioni anomale, che potranno essere risolte o comprese anche tramite consulenza al team di sviluppo.

Le eccezioni saranno inoltre suddivise in eccezioni causate da errori di validazione, comunicazione e persistenza.

### **Documentazione delle interfacce**

Ogni metodo include una breve documentazione che ne descrive il comportamento atteso, pre e post-condizioni e eventuali eccezioni che potrebbero essere sollevate.

## 2. Pacchetti

Questa sezione descrive la decomposizione dei subsystems in package software e l'organizzazione dei file del codice sorgente per il sistema *Easy Work Management System* (E.W.M.S.).

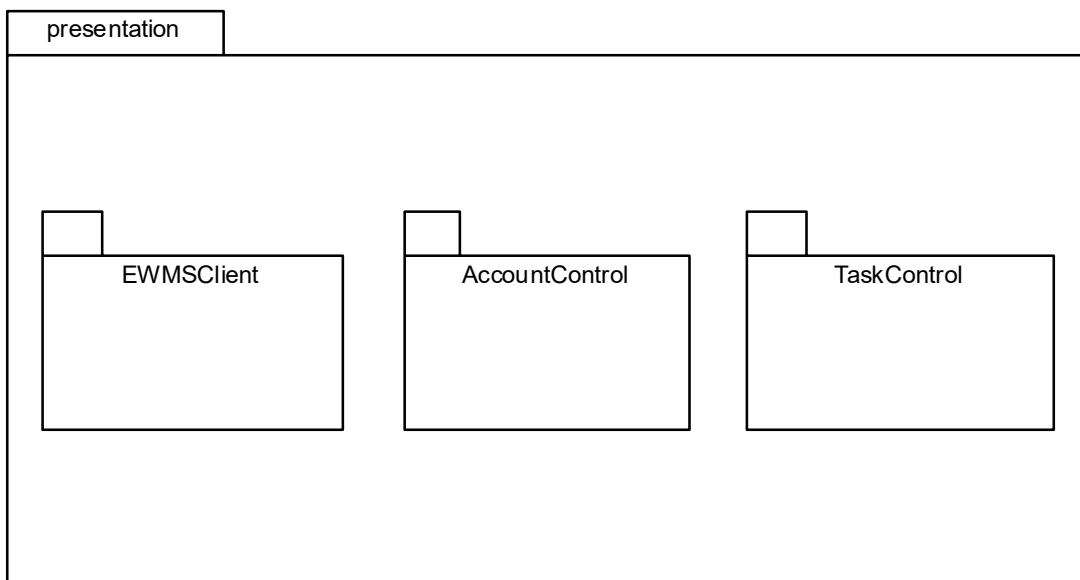
L'organizzazione dei package segue l'architettura Three-Tier definita nel System Design Document, separando chiaramente la logica di presentazione, la logica di business e la persistenza dei dati. Il codice sorgente Java è organizzato sotto il root package **it.unisa.ewms**. La struttura riflette la suddivisione logica in layer e l'adozione del pattern Model-View-Controller (MVC).

I principali gruppi di package identificati sono:

- Presentation package
- Application package
- Model package
- Persistence package

### 2.1 Decomposizione dei sottosistemi in package

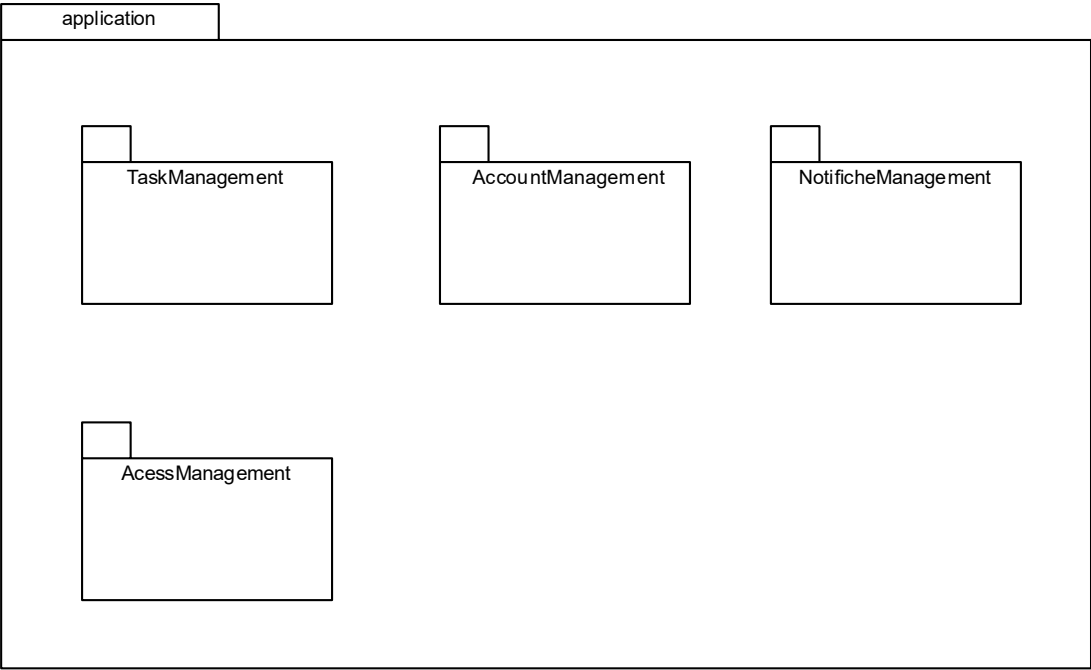
#### **it.unisa.ewms.presentation**



Questo package implementa il Presentation Layer e corrisponde ai componenti EWMSClient, AccountControl e TaskControl. Include le classi Servlet che fungono da Controller (nei

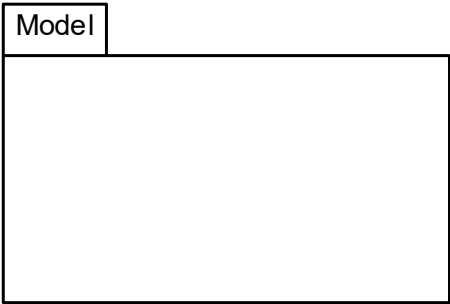
sottopacchetti AccountControl e TaskControl) e JSP che fungono da vista (nel sottopacchetto EWMSClient).

**it.unisa.ewms.application**



Questo package implementa l'Application Layer e raggruppa i servizi di business logic AccessManagement, TaskManagement, NotificheManagement e AccountManagement, ognuno dei quali è racchiuso nel corrispondente sottopacchetto.

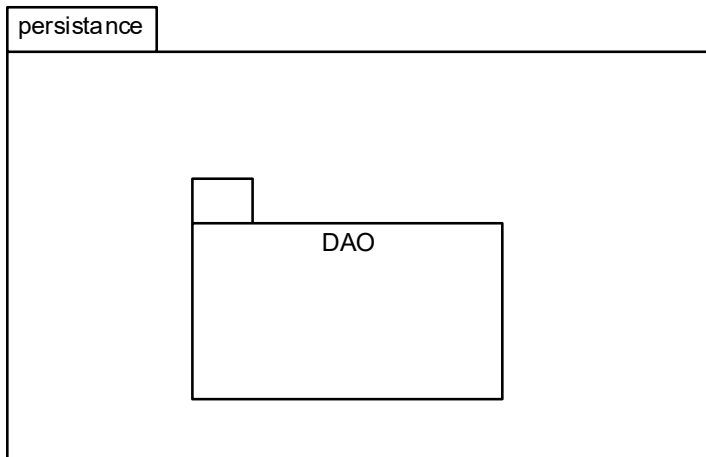
**it.unisa.ewms.model**





Questo package rappresenta i dati trasversali a tutti i livelli e corrisponde alle entità definite nel RAD. Contiene quindi i bean che verranno utilizzati come mezzo di trasporto delle informazioni.

### it.unisa.ewms.persistence



Questo package implementa il Persistence Layer e il componente PersistenceManagement. Implementa il pattern DAO (Data Access Object). Gestisce la connessione al DBMS MySQL tramite JDBC, l'esecuzione delle query SQL e il mapping dei ResultSet in oggetti del Model.

## 2.3 Dipendenze tra package

Le dipendenze tra i package sono regolate rigorosamente per evitare dipendenze cicliche e rispettare la separazione dei compiti.

- Flusso di Dipendenza:
  1. *presentation* dipende da *application* e *model*. I Controller invocano i metodi dei servizi per eseguire operazioni.
  2. *application* dipende da *storage* e *model*. I servizi coordinano le chiamate ai DAO per recuperare o salvare dati.
  3. *persistence* dipende solo da *model* e dalle librerie JDBC. Non ha conoscenza della logica di business o della presentazione.
  4. *model* è indipendente e non ha dipendenze verso gli altri package del sistema.

### 3. Class interfaces

Questa sezione del documento descrive in dettaglio le Class interfaces progettate per il sistema Easy Work Management System (E.W.M.S.). L'obiettivo è fornire una specifica tecnica completa delle componenti software, definendo i contratti pubblici che regolano le interazioni tra gli oggetti del sistema.

#### 3.1 AccessManagement

##### 3.1.1 SessionService

|                        |                                                                                                                                                                                                                                                 |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Nome classe</b>     | SessionService                                                                                                                                                                                                                                  |
| <b>Descrizione</b>     | Classe di servizio che incapsula la logica di business relativa all'autenticazione e all'autorizzazione. Interagisce con il persistence layer (tramite CatalogoUtenti) per la verifica delle credenziali e gestisce lo stato della HttpSession. |
| <b>Metodi</b>          | + login(username: String, password: String): void<br>+ logout(currentSession: HttpSession): void<br>+ checkPermission(user: Utente, requiredRole: Role): boolean                                                                                |
| <b>Invariante</b>      | /                                                                                                                                                                                                                                               |
|                        |                                                                                                                                                                                                                                                 |
| <b>Firma metodo</b>    | + login(username: String, password: String): void                                                                                                                                                                                               |
| <b>Descrizione</b>     | Verifica la corrispondenza delle credenziali fornite interagendo con il catalogo utenti. In caso di successo, inizializza la sessione utente.                                                                                                   |
| <b>Pre-condizione</b>  | Context:SessionService::login<br>username != null AND password != null.                                                                                                                                                                         |
| <b>Post-condizione</b> | Context:SessionService::login<br>HttpSession.getParameter("utente") != null                                                                                                                                                                     |
|                        |                                                                                                                                                                                                                                                 |
| <b>Firma metodo</b>    | + logout(currentSession: HttpSession): void                                                                                                                                                                                                     |
| <b>Descrizione</b>     | Rimuove tutti gli attributi di sessione e la invalida lato server.                                                                                                                                                                              |
| <b>Pre-condizione</b>  | Context:SessionService::logout<br>HttpSession.getParamenter("utente") != null                                                                                                                                                                   |

|                        |                                                                                                                                                       |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Post-condizione</b> | Context:SessionService::logout<br>HttpSession.getParamenter("utente") == null                                                                         |
|                        |                                                                                                                                                       |
| <b>Firma metodo</b>    | + checkPermission(utente: Utente, requiredRole: String): boolean                                                                                      |
| <b>Descrizione</b>     | Verifica se l'utente possiede i permessi necessari per eseguire una specifica azione, supportando il controllo degli accessi basato sui ruoli (RBAC). |
| <b>Pre-condizione</b>  |                                                                                                                                                       |
| <b>Post-condizione</b> | Ritorna true se il ruolo dell'utente corrisponde a requiredRole                                                                                       |

## 3.2 AccountManagement

### 3.2.1 *PersonaProfileService*

|                             |                                                                                                                                                       |
|-----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Nome Classe</b>          | PersonalProfileService                                                                                                                                |
| <b>Descrizione</b>          | gestisce tutte le operazioni che un utente autenticato può eseguire sul proprio account                                                               |
| <b>Metodi</b>               | + getMyProfile(ID: string) : Utente<br>+ changePwd(ID: string, oldpwd: string, newpwd: string) : bool                                                 |
| <b>Invariante di classe</b> | /                                                                                                                                                     |
|                             |                                                                                                                                                       |
| <b>Nome Metodo</b>          | + getMyProfile(id: int): Utente                                                                                                                       |
| <b>Descrizione</b>          | Recupera i dettagli dell'account dell'utente correntemente loggato                                                                                    |
| <b>Pre-condizione</b>       | Context:PersonalProfileService::getMyProfile(ID: string)<br><br>Pre:<br><br>result <> null and<br>result.isActive = true and<br>result.utente.id = ID |
| <b>Post-condizione</b>      | Context:PersonalProfileService::getMyProfile(ID: string)                                                                                              |

|                        |                                                                                                                                                                                                                                                                                     |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                        | Post:<br>result $\neq$ null and<br>result.utente = Account.allInstances()<br>-> select(a   a.id = ID) and<br>.currentView = View::Profilo                                                                                                                                           |
|                        |                                                                                                                                                                                                                                                                                     |
| <b>Nome Metodo</b>     | + changePwd(id: int, oldpwd: string, newpwd: string) : bool                                                                                                                                                                                                                         |
| <b>Descrizione</b>     | Permette all'utente di aggiornare la propria password.                                                                                                                                                                                                                              |
| <b>Pre-condizione</b>  | Context:PersonalProfileService::changePwd(ID: string, oldpwd: string, newpwd: string)<br><br>Pre:<br>Account.allInstances() -> any(a   a.id = ID)<br>.currentView = View::Profile                                                                                                   |
| <b>Post-condizione</b> | Context:PersonalProfileService::changePwd(ID: string, oldpwd: string, newpwd: string)<br><br>Post:<br>Account.allInstances() -> select(a   a.id = ID)<br>.currentView = View::SchedaModificaPwd and<br>Account.allInstances() -> any(a   a.id = ID)<br>.feedbackMessage $\neq$ null |

### 3.2.2 ProfileManagementService

|                      |                                                                                                                                                                                                                                                                           |
|----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Nome Classe          | ProfileManagementService                                                                                                                                                                                                                                                  |
| Descrizione          | gestisce le operazioni amministrative riservate al ruolo del Gestore degli account                                                                                                                                                                                        |
| Metodi               | + addAccount(account: Utente) : bool<br>+ getAccount(ID: string) : Utente<br>+ generateNewPwd(account: Utente) : string<br>+ modifyRole(account: Utente, newRole: string) : bool<br>+ modifySupervisor(account: Utente) : bool<br>+ deleteAccount(account: Utente) : bool |
| Invariante di classe | /                                                                                                                                                                                                                                                                         |
|                      |                                                                                                                                                                                                                                                                           |
| Nome Metodo          | + addAccount(Utente) : bool                                                                                                                                                                                                                                               |
| Descrizione          | Permette di aggiungere un nuovo account al sistema                                                                                                                                                                                                                        |

|                 |                                                                                                                                                                                                                                  |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pre-condizione  | Context:ProfileManagementService::addAccount(account: Utente)<br><br>Pre:<br><br>Session.utente.isAuthenticated() = true and<br>Session.utente.ruolo = role::accountManager and<br>.currentView = View::creaAccount              |
| Post-condizione | Context:ProfileManagementService::addAccount(account: Utente)<br><br>Post:<br>result <> null and<br>Account.allInstances() -> includes(result) and<br>result.nomeUtente = account.nomeUtente and<br>result.ruolo = account.ruolo |
|                 |                                                                                                                                                                                                                                  |
| Nome Metodo     | + getAccount(ID: string) : Utente                                                                                                                                                                                                |
| Descrizione     | Permette di recuperare i dati di un account selezionato dal client.                                                                                                                                                              |
| Pre-condizione  | Context:ProfileManagementService::getAccount(ID: string)<br><br>Pre:<br><br>Session.utente.isAuthenticated() = true and<br>Session.utente.ruolo = role::accountManager and<br>.currentView = View::Homepage                      |
| Post-condizione | Context:ProfileManagementService::getAccount(ID: string)<br><br>Post:<br>result <> null and<br>result.id = ID and<br>.currentView = View::Profilo                                                                                |
|                 |                                                                                                                                                                                                                                  |
| Nome Metodo     | + generateNewPwd(account: Utente) : string                                                                                                                                                                                       |
| Descrizione     | Permette di rimpiazzare la password preesistente di un account con una nuova.                                                                                                                                                    |
| Pre-condizione  | Context:ProfileManagementService::generateNewPwd(account: Utente)<br><br>Pre:<br><br>Session.utente.isAuthenticated() = true and<br>Session.utente.ruolo = role::accountManager and<br>.currentView = View::Profile(Utente)      |
| Post-condizione | Context:ProfileManagementService:: generateNewPwd(account: Utente)<br><br>Post:<br>result <> null and                                                                                                                            |

|                 |                                                                                                                                                                                                                                                 |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                 | Account.allInstances() -> select(a   a.id = account.id).password <><br>Account.allInstances() -> select(a   a.id = account.id).password@pre<br>and<br>.currentView = View::Profile(Utente)                                                      |
| Nome Metodo     | + modifyRole(account: Utente, newRole: string) : bool                                                                                                                                                                                           |
| Descrizione     | Permette di cambiare il ruolo assegnato ad un account.                                                                                                                                                                                          |
| Pre-condizione  | Context:ProfileManagementService::modifyRole(account: Utente, newRole: string)<br><br>Pre:<br><br>Session.utente.isAuthenticated() = true and<br>Session.utente.ruolo = role::accountManager and<br>.currentView = View::Profile(Utente)        |
| Post-condizione | Context:ProfileManagementService:: modifyRole(account: Utente newRole: string)<br><br>Post:<br>result <> null and<br>result.value = true or result.value = false and<br>account.ruolo = newRole and<br>.currentView = View::Profile(Utente)     |
| Nome Metodo     | + modifySupervisor(account: Utente, newSup: string) : bool                                                                                                                                                                                      |
| Descrizione     | Permette di cambiare supervisore ad un dipendente.                                                                                                                                                                                              |
| Pre-condizione  | Context:ProfileManagementService:: modifySupervisor(account: Utente, newSup: string)<br><br>Pre:<br><br>Session.utente.isAuthenticated() = true and<br>Session.utente.ruolo = role::accountManager and<br>.currentView = View::Profile(Utente)  |
| Post-condizione | Context:ProfileManagementService:: modifyRole(account: Utente newSup: string)<br><br>Post:<br>result <> null and<br>result.value = true or result.value = false and<br>account.supervisore = newSup and<br>.currentView = View::Profile(Utente) |
| Nome Metodo     | + deleteAccount(account: Utente) : bool                                                                                                                                                                                                         |
| Descrizione     | Permette di cambiare supervisore ad un dipendente.                                                                                                                                                                                              |

|                 |                                                                                                                                                                                                                                                                 |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pre-condizione  | Context:ProfileManagementService:: deleteAccount(account: Utente)<br><br>Pre:<br><br>Session.utente.isAuthenticated() = true and<br>Session.utente.ruolo = role::accountManager and<br>.currentView = View::Profile(Utente)                                     |
| Post-condizione | Context:ProfileManagementService:: deleteAccount(account: Utente)<br><br>Post:<br><br>Notifiche.allInstances() -> any(a   a.id = account.id) -> isEmpty() and<br>Account.allInstances() -> excludes(a   a.id = account.id) and<br>.currentView = View::Homepage |

### 3.3 TaskManagement

#### 3.3.1 TaskCommonService

|                      |                                                                                                                                                         |
|----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| Nome Classe          | TaskCommonService                                                                                                                                       |
| Descrizione          | espone le operazioni di lettura e le azioni condivise accessibili sia ai Dipendenti che ai Supervisor                                                   |
| Metodi               | + getTask(taskID: int) : Task<br>+ filterTask(filter: string) : List<Task><br>+ holdTask(taskID: int) : bool<br>+ downloadAllegato(taskID) : FileStream |
| Invariante di classe | /                                                                                                                                                       |
|                      |                                                                                                                                                         |
| Nome Metodo          | + getTask(taskID: int) : Task                                                                                                                           |
| Descrizione          | Recupera le informazioni complete di un task.                                                                                                           |
| Pre-condizione       | Context:TaskCommonService::getTask(taskID: int)<br><br>Pre:<br><br>Session.utente.isAuthenticated() = true and<br>.currentView = View::Homepage         |
| Post-condizione      | Context:TaskCommonService::getTask(taskID: int)                                                                                                         |

|                 |                                                                                                                                                                                                                                                                 |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                 | Post:<br><br>result <> null and<br>result.id = taskID and<br>.currentView = View::Task(taskID)                                                                                                                                                                  |
| Nome Metodo     | + filterTask(filter: string) : List<Task>                                                                                                                                                                                                                       |
| Descrizione     | Permette di filtrare task per stato (da completare, inizializzate ecc...).                                                                                                                                                                                      |
| Pre-condizione  | Context:TaskCommonService::filterTask(filter: string)<br><br>Pre:<br><br>Session.utente.isAuthenticated() = true and<br>.currentView = View::Homepage                                                                                                           |
| Post-condizione | Context:TaskCommonService::filterTask(filter: string)<br><br>Post:<br><br>result <> null and<br>result -> forAll(t   t.status = filter) and<br>.currentView = View::Homepage                                                                                    |
| Nome Metodo     | + holdTask(taskID: int) : bool                                                                                                                                                                                                                                  |
| Descrizione     | Permette di cambiare sospendere un task e cambiare di conseguenza lo stato.                                                                                                                                                                                     |
| Pre-condizione  | Context:TaskCommonService::holdTask(taskID: int)<br><br>Pre:<br><br>Session.utente.isAuthenticated() = true and<br>.currentView = View::Task(taskID) and<br>Task.allInstances() -> select(t   t.id = taskID).status <> status::sospeso<br>or status::completato |
| Post-condizione | Context:TaskCommonService::holdTask(taskID: int)<br><br>Post:<br><br>result <> null and<br>result.value = true and<br>Task.allInstances() -> select(t   t.id = taskID).status = sospeso                                                                         |
| Nome Metodo     | + downloadAllegato(taskID: int) : FileStream                                                                                                                                                                                                                    |
| Descrizione     | Permette di scaricare un allegato dal task in cui è stato caricato.                                                                                                                                                                                             |
| Pre-condizione  | Context:TaskCommonService::downloadAllegato(taskID: int)<br><br>Pre:                                                                                                                                                                                            |



|                 |                                                                                                                                                                 |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                 | Session.utente.isAuthenticated() = true and<br>.currentView = View::Task(taskID) and<br>Task.allInstances() -> select(t   t.id = taskID).allegati -> notEmpty() |
| Post-condizione | Context:TaskCommonService::downloadAllegato(taskID: int)<br><br>Post:<br><br>result <> null and<br>.currentView = View::Task(taskID)                            |

### 3.3.2 TaskSupervisoreService

|                      |                                                                                                                                                                                                                                                                                                                                                              |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Nome Classe          | TaskSupervisoreService                                                                                                                                                                                                                                                                                                                                       |
| Descrizione          | espone tutte le operazioni eseguibili da un Supervisore sul ciclo di vita di un task                                                                                                                                                                                                                                                                         |
| Metodi               | + createTask(taskID: int, descrizione: string, dataScadenza: LocalDate, priority: string) : Task<br><br>+ deleteTask(taskID: int) : bool<br><br>+ sendWarning(taskID: int, msg: string) : bool                                                                                                                                                               |
| Invariante di classe | /                                                                                                                                                                                                                                                                                                                                                            |
|                      |                                                                                                                                                                                                                                                                                                                                                              |
| Nome Metodo          | + createTask(taskID: int, descrizione: string, dataScadenza: LocalDate, priority: string, dipendenteID: string) : Task                                                                                                                                                                                                                                       |
| Descrizione          | Permette di assegnare un task ad un dipendente.                                                                                                                                                                                                                                                                                                              |
| Pre-condizione       | Context:taskSupervisoreService::createTask(taskID: int, descrizione: string, dataScadenza: LocalDate, priority: string, dipendenteID: string)<br><br>Pre:<br><br>Session.utente.isAuthenticated() = true and<br>Session.utente.ruolo = role::Supervisore<br>Account.allInstances() -> exists(a   a.id = dipendenteID) and<br>.currentView = View::createTask |
| Post-condizione      | Context:taskSupervisoreService::createTask(taskID: int, descrizione: string, dataScadenza: LocalDate, priority: string, dipendenteID: string)<br><br>Post:<br><br>result <> null and<br>Task.allInstances() -> size() += 1 and                                                                                                                               |

|                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                 | Task.allInstances() -> select(t   t.id = result.id).dipendenteId =<br>dipendenteId and<br>Task.allInstances() -> select(t   t.id = result.id).status = status::InCorso<br>and<br>.currentView = View::Task(taskID)                                                                                                                                                                                                                                  |
| Nome Metodo     | + deleteTask(taskID: int) : bool                                                                                                                                                                                                                                                                                                                                                                                                                    |
| Descrizione     | Permette di rimuovere un task precedentemente assegnato dal sistema e notifica il dipendente.                                                                                                                                                                                                                                                                                                                                                       |
| Pre-condizione  | Context:TaskSupervisoreService:: deleteTask(taskID: int)<br><br>Pre:<br><br>Session.utente.isAuthenticated() = true and<br>.currentView = View::Task(taskID) and<br>Task.allInstances() -> exists(t   t.id = taskID)                                                                                                                                                                                                                                |
| Post-condizione | Context:TaskSupervisoreService::deleteTask(taskID; int)<br><br>Post:<br><br>result <> null and<br>result.value = true and<br>Task.allInstances() -> excludes(t   t.id = taskID) and<br>Notification.allInstances() -> size() += 1 and<br>.currentView = View::Homepage                                                                                                                                                                              |
| Nome Metodo     | + sendWarning(taskID: int, msg: string) : bool                                                                                                                                                                                                                                                                                                                                                                                                      |
| Descrizione     | Invia un sollecito o un avviso formale al dipendente assegnato al task.                                                                                                                                                                                                                                                                                                                                                                             |
| Pre-condizione  | Context:TaskSupervisoreService:: sendWarning(taskID: int, msg: string)<br><br>Pre:<br><br>Session.utente.isAuthenticated() = true and<br>Session.utente.ruolo = role::Supervisore<br>.currentView = View::Task(taskID)                                                                                                                                                                                                                              |
| Post-condizione | Context:TaskSupervisoreService:: sendWarning(taskID: int, msg: string)<br><br>Post:<br>result <> null and<br>result.value = true and<br>Notifiche.allInstances() -> size() += 1 and<br><u>Notifiche.allInstances() -&gt; select(n   n.dipendenteId =</u><br><u>Task.allInstances() -&gt; select(t   t.id = taskID).dipendenteId) and</u><br><u>Notifiche.allInstances() -&gt; select(n   n.body = msg) and</u><br>.currentView = View::Task(taskID) |

### 3.3.3 TaskDipendenteService

|                      |                                                                                                                                                                                                                                                                                                                                                                                             |
|----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Nome Classe          | TaskDipendenteService                                                                                                                                                                                                                                                                                                                                                                       |
| Descrizione          | espone tutte le operazioni eseguibili da un Dipendente sul ciclo di vita di un task.                                                                                                                                                                                                                                                                                                        |
| Metodi               | + inizializzaTask(taskID: int) : bool<br><br>+ completeTask(taskID: int) : bool                                                                                                                                                                                                                                                                                                             |
| Invariante di classe | /                                                                                                                                                                                                                                                                                                                                                                                           |
|                      |                                                                                                                                                                                                                                                                                                                                                                                             |
| Nome Metodo          | + inizializzaTask(taskID: int) : bool                                                                                                                                                                                                                                                                                                                                                       |
| Descrizione          | Segnala l'inizio dei lavori da parte di un dipendente, portando il task dallo stato "Da completare" a "In elaborazione".                                                                                                                                                                                                                                                                    |
| Pre-condizione       | Context:TaskDipendenteService::inizializzaTask(taskID: int)<br><br>Pre:<br><br>Session.utente.isAuthenticated() = true and<br>Session.utente.ruolo = role::Dipendente and<br>Session.utente.id = Task.allInstances() -> select(t   t.id = taskID).dipendenteId and<br>Task.allInstances()->select(t   t.id = taskID).status = status::DaCompletare and<br>.currentView = View::Task(taskID) |
| Post-condizione      | Context:TaskDipendenteService::inizializzaTask(taskID: int)<br><br>Post:<br><br>result <> null and<br>result.value = true and<br>Task.allInstances() -> select(t   t.id = taskID).status = status::InElaborazione                                                                                                                                                                           |
|                      |                                                                                                                                                                                                                                                                                                                                                                                             |
| Nome Metodo          | + completeTask(taskID: int) : bool                                                                                                                                                                                                                                                                                                                                                          |
| Descrizione          | Segnala la conclusione del lavoro, portando il task nello stato finale "Completato".                                                                                                                                                                                                                                                                                                        |
| Pre-condizione       | Context:TaskDipendenteService::completeTask(taskID: int)<br><br>Pre:<br><br>Session.utente.isAuthenticated() = true and<br>Session.utente.ruolo = role::Dipendente and                                                                                                                                                                                                                      |

|                 |                                                                                                                                                                                                                                                                                                                                          |
|-----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                 | <pre> Sessione.utente.id = Task.allInstances() -&gt; select(t   t.id = taskID).dipendenteId and Task.allInstances()-&gt;select(t   t.id = taskID).status = status::InElaborazione and .currentView = View::Task(taskID) </pre>                                                                                                           |
| Post-condizione | <pre> Context:TaskDipendenteService::completeTask(taskID: int)  Post:  result &lt;&gt; null and result.value = true and Task.allInstances() -&gt; select(t   t.id = taskID).status = status::Completato and Notifiche.allInstances -&gt; size() += 1 and Notifiche.allInstances() -&gt; select(n   n.taskId = taskID).isDefined() </pre> |

## 3.4 NotificheManagement

### 3.4.1 NotificheService

|                 |                                                                                                                                           |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Nome classe     | NotificheService                                                                                                                          |
| Descrizione     | Classe che serve per la gestione delle notifiche generate dai task del sistema e per la visualizzazione da parte del cliente              |
| Metodi          | <pre> +showNotification(utente:Utente):List&lt;Notifica&gt;  +send(sender:String, receiver:String, message:String, type:enum):void </pre> |
| Invariante      | \                                                                                                                                         |
| Firma metodo    | +showNotification(utente:Utente):List<Notifica>                                                                                           |
| Descrizione     | Serve per recuperare le notifiche ricevute da parte di un utente                                                                          |
| Pre-condizione  | <pre> Context::NotificheService:showNotification(utente:Utente): List&lt;Notifica&gt;  utente != null </pre>                              |
| Post-condizione | <pre> NotificationController::showNotification(utente:Utente): List&lt;Notifica&gt;  result = utente.notifications </pre>                 |

|                 |                                                                                                                                                                                       |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Firma metodo    | +send(sender:String, receiver:String, message:String, type:enum):void                                                                                                                 |
| Descrizione     | Implementa la logica di invio di una notifica. Crea una nuova istanza di notifica, la salva nel sistema persistente e tenta l'invio immediato al client del destinatario se connesso. |
| Pre-condizione  | Context::NotificheService:send(sender:String, receiver:String, message:String, type:enum):void<br><br>sender != null AND receiver != null AND message != null                         |
| Post-condizione | \                                                                                                                                                                                     |

## 3.5 PersistenceManagement

### 3.5.1 PersistenceService

|                 |                                                                                                     |
|-----------------|-----------------------------------------------------------------------------------------------------|
| Nome classe     | PersistenceService                                                                                  |
| Descrizione     | Fornisce l'accesso ai DAO specifici per la gestione delle entità                                    |
| Metodi          | + getUtenteDAO(): UtenteDAO<br><br>+ getTaskDAO(): TaskDAO<br><br>+ getNotificheDAO(): NotificheDAO |
| Invariante      | \                                                                                                   |
|                 |                                                                                                     |
| Firma metodo    | + getUtenteDAO()                                                                                    |
| Descrizione     | Restituisce l'istanza del DAO per la gestione degli utenti                                          |
| Pre-condizione  | \                                                                                                   |
| Post-condizione | Context PersistenceService::getUtenteDAO()<br><br>result.oclIsKindOf(UtenteDAO) AND result <> null  |
|                 |                                                                                                     |
| Firma metodo    | + getTaskDAO()                                                                                      |
| Descrizione     | Restituisce l'istanza del DAO per la gestione dei task                                              |

|                 |                                                                                                          |
|-----------------|----------------------------------------------------------------------------------------------------------|
| Pre-condizione  | \                                                                                                        |
| Post-condizione | Context PersistenceService::getTaskDAO()<br>result.oclIsKindOf(TaskDAO) AND result $\neq$ null           |
|                 |                                                                                                          |
| Firma metodo    | + getNotificheDAO()                                                                                      |
| Descrizione     | Restituisce l'istanza del DAO per la gestione delle notifiche                                            |
| Pre-condizione  | \                                                                                                        |
| Post-condizione | Context PersistenceService::getNotificheDAO()<br>result.oclIsKindOf(NotificheDAO) and result $\neq$ null |

### 3.5.2 *UtenteDAO*

|                 |                                                                                                                                          |
|-----------------|------------------------------------------------------------------------------------------------------------------------------------------|
| Nome classe     | UtenteDAO                                                                                                                                |
| Descrizione     | Gestisce la persistenza per l'entità Utente e le sue sottoclassi                                                                         |
| Metodi          | + create(utente:Utente) : void<br>+ findByEmail(email:String) : Utente<br>+ update(utente:Utente) : void<br>+ delete(utente:Utente):void |
| Invariante      | \                                                                                                                                        |
|                 |                                                                                                                                          |
| Firma metodo    | +create(utente:Utente) : void                                                                                                            |
| Descrizione     | Persiste un nuovo utente nel database                                                                                                    |
| Pre-condizione  | Context UtenteDAO::create(utente:Utente)<br>utente $\neq$ null                                                                           |
| Post-condizione | Context UtenteDAO::create(utente:Utente)<br>Utente.allInstances()->includes(u)                                                           |
|                 |                                                                                                                                          |
| Firma metodo    | + findByEmail(email:String) : Utente                                                                                                     |

|                 |                                                                                                                    |
|-----------------|--------------------------------------------------------------------------------------------------------------------|
| Descrizione     | Recupera un utente tramite la sua email                                                                            |
| Pre-condizione  | Context UtenteDAO::findByEmail(email:String)<br>Email <> null                                                      |
| Post-condizione | Context UtenteDAO::findByEmail(email:String)<br>result = Utente.allInstances()->any(u   u.email = email)           |
|                 |                                                                                                                    |
| Firma metodo    | + update(utente:Utente) : void                                                                                     |
| Descrizione     | Aggiorna un'istanza di utente già presente nel database                                                            |
| Pre-condizione  | Context UtenteDAO::update(utente:Utente)<br>Utente.allInstances()->exists(x   x.email = utente.email)              |
| Post-condizione | Context UtenteDAO::update(utente:Utente)<br>Utente.allInstances()->select(x   x.email = utente.email)-> x = utente |
|                 |                                                                                                                    |
| Firma metodo    | + delete(utente:Utente):void                                                                                       |
| Descrizione     | Elimina l'utente specificato dal database                                                                          |
| Pre-condizione  | Context UtenteDAO:: delete(utente:Utente)<br>Utente.allInstances()->includes(utente)                               |
| Post-condizione | Context UtenteDAO:: delete(utente:Utente)<br>Utente.allInstances()->excludes (utente)                              |

### 3.5.3 TaskDAO

|             |                                                                                                      |
|-------------|------------------------------------------------------------------------------------------------------|
| Nome classe | TaskDAO                                                                                              |
| Descrizione | Gestisce la persistenza per l'entità Task                                                            |
| Metodi      | + create (task:Task):void<br>+ findById(id:int) : Task<br>+ findByUtente(utente:Utente) : List<Task> |

|                 |                                                                                                                                           |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------|
|                 | + update(task:Task) : void<br>+ delete (task:Task) : void                                                                                 |
| Invariante      | \                                                                                                                                         |
|                 |                                                                                                                                           |
| Firma metodo    | + create (task:Task):void                                                                                                                 |
| Descrizione     | Persistente una task all'interno del database                                                                                             |
| Pre-condizione  | Context TaskDAO::create(task: Task)<br>Task.allInstances()->forall(x   x.id <> task.id)                                                   |
| Post-condizione | Context TaskDAO::create(task: Task)<br>Task.allInstances()->includes(t)                                                                   |
|                 |                                                                                                                                           |
| Firma metodo    | + findById(id:int) : Task                                                                                                                 |
| Descrizione     | Trova un task in base al suo id                                                                                                           |
| Pre-condizione  | Context TaskDAO::findById(id:int)<br>Id <> null                                                                                           |
| Post-condizione | Context TaskDAO::findById(id:int)<br>Task.allInstances()->select(t   t.id = id)                                                           |
|                 |                                                                                                                                           |
| Firma metodo    | + findByUtente(utente:Utente) : List<Task>                                                                                                |
| Descrizione     | Ritorna una lista di tutte i task assegnati ad un utente                                                                                  |
| Pre-condizione  | Context TaskDAO::findByUtente(utente:Utente)<br>utente <> null                                                                            |
| Post-condizione | Context TaskDAO::findByUtente(utente:Utente)<br>result = Task.allInstances()->select(t   t.dipendente = utente or t.supervisore = utente) |
|                 |                                                                                                                                           |
| Firma metodo    | + update(task:Task) : void                                                                                                                |
| Descrizione     | Aggiorna un task all'interno del database                                                                                                 |
| Pre-condizione  | Context TaskDAO::update(task: Task)<br>Task.allInstances()->exists(x   x.id = task.id)                                                    |
| Post-condizione | Context TaskDAO::update(task: Task)<br>Utente.allInstances()->select(x   x.id = task.id) -> x = task                                      |
|                 |                                                                                                                                           |
| Firma metodo    | + delete (task:Task) : void                                                                                                               |
| Descrizione     | Elimina un task dal database                                                                                                              |
| Pre-condizione  | Context TaskDAO::delete(task: Task)<br>Task.allInstances()->includes(t)                                                                   |



|                 |                                                                                                                                        |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------|
| Post-condizione | Context TaskDAO::delete(task: Task)<br><br>Task.allInstances()->excludes (t) AND Allegato.allInstances()->forAll(a   a.taskId <> t.id) |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------|

### 3.5.4 NotificheDAO

|                 |                                                                                                                              |
|-----------------|------------------------------------------------------------------------------------------------------------------------------|
| Nome classe     | NotificaDAO                                                                                                                  |
| Descrizione     | Gestisce la persistenza per l'entità Notifica                                                                                |
| Metodi          | + create(notifica:Notifica):void<br><br>+ findByUtente(utente:Utente): List<Notifica>                                        |
| Invariante      | \                                                                                                                            |
|                 |                                                                                                                              |
| Firma metodo    | + create(notifica:Notifica):void                                                                                             |
| Descrizione     | Inserisce all'interno del database una notifica                                                                              |
| Pre-condizione  | Context NotificaDAO::create(notifica: Notifica)<br>notifica <> null                                                          |
| Post-condizione | Context NotificaDAO::create(notifica: Notifica)<br>Notifica.allInstances()->includes(notifica)                               |
|                 |                                                                                                                              |
| Firma metodo    | + findByUtente(utente:Utente): List<Notifica>                                                                                |
| Descrizione     | Carica dal database tutte le notifiche dell'utente specificato                                                               |
| Pre-condizione  | Context NotificaDAO::findByUtente(utente: Utente)<br>utente <> null                                                          |
| Post-condizione | Context NotificaDAO::findByUtente(utente: Utente)<br>result = Notifica.allInstances()->select(n   n.receiver = utente.email) |